

Лабораторная работа №2

Анализ защищённости приложений и программных систем

Части: 2.1 — оценка защищённости разработанного приложения; 2.2 — оценка защищённости сторонних программных систем

Приложение: `pziis-vault` — веб-реестр с разделением данных на конфиденциальные и неконфиденциальные

Стек: Python 3.12 · FastAPI · PostgreSQL 15 · nginx · Docker

Объём: 2656 строк кода и схемы, 56 автоматических проверок, 12 сценариев демонстрации

Этот документ: объяснение каждой темы простым языком, что реализовано и ответы на вопросы

1. О чём эта работа простыми словами	2
2. Два класса данных — основа всего	2
3. Криптография простыми словами	4
4. Слепой индекс: как искать в зашифрованном	7
5. Как устроен вход и как работает сессия	8
6. Атаки на веб-приложение и защита от них	10
7. Два контура контроля доступа	12
8. Минимальные привилегии: в СУБД и в ОС	13
9. Защита канала связи	14
10. Что реализовано	14
11. Вопросы и ответы	16
12. Итог	18

1. О чём эта работа простыми словами

Первая работа отвечала на вопрос «кому что разрешено». Вторая — на вопрос «а выдержит ли это нападение».

Разница принципиальная. Настроить права — значит описать нормальное поведение системы. Проанализировать защищённость — значит представить себе злоумышленника и проверить, что он не сможет сделать. Методичка называет этот подход анализом по принципу **чёрного ящика**: система изучается снаружи, как её видит атакующий.

Задание требует пять функций и анализ по четырём направлениям. Пять функций — это обычное приложение: регистрация, вход, работа с данными, выход. Четыре направления анализа — вот они, и их стоит запомнить, потому что тот же каркас используется и в части 2.2:

Направление	Простыми словами
Политика безопасности операционной системы	с какими правами работает сам процесс приложения и что он может испортить, если его взломают
Политика безопасности компонентов и их взаимодействия	как приложение общается с базой данных и прокси-сервером, и что увидит атакующий, пробравшийся в одно звено
Сетевая политика безопасности	какие порты открыты наружу, шифруется ли трафик
Защищённость конфиденциальных данных	что произойдёт, если украдут копию базы данных

2. Два класса данных — основа всего

Задание требует, чтобы приложение работало и с конфиденциальными, и с неконфиденциальными данными. Это не украшение: вокруг этого разделения строится вся работа.

	Неконфиденциальные	Конфиденциальные
Пример	публичный регламент, объявление	реквизиты договора, персональные данные
Как хранятся	открытым текстом	зашифрованными
Как ищутся	обычный полнотекстовый поиск	по «слепому индексу» — см. раздел 4
Кто видит	публичные — все, включая гостя	только владелец записи
Что будет при краже базы	прочитают всё	получат только шифротекст

ПОЧЕМУ ЭТО ВАЖНО ПРОГОВОРИТЬ

Разделение доведено до **уровня хранения**: два разных класса лежат в разных таблицах, под разными привилегиями, с разными механизмами поиска и разными политиками доступа. Их нельзя перепутать «по невнимательности» — структура не позволяет. Если бы оба класса лежали в одной таблице и различались только пометкой, всякая ошибка в коде приводила бы к утечке.

3. Криптография простыми словами

3.1. Хеширование и шифрование — разные инструменты

Это первое, что нужно разделять. Их путают чаще всего.

	Хеширование	Шифрование
Направление	только вперёд, обратно нельзя	туда и обратно
Ключ	не нужен	нужен
Зачем	сравнить, не зная исходного значения	спрятать, но потом прочитать
Где в работе	пароли, токены сессий	содержимое конфиденциальных записей

Правило выбора простое. Если значение нужно будет показать пользователю — шифруем. Если достаточно уметь проверить «то же самое или нет» — хешируем, это надёжнее, потому что нечего расшифровывать.

Поэтому пароли **хешируются**: приложению не нужно знать пароль, ему нужно лишь проверить введённый. А текст договора **шифруется**: его надо будет показать владельцу.

3.2. Почему пароли нельзя просто прогнать через SHA-256

SHA-256 — хорошая хеш-функция, но она спроектирована **быстрой**. Это её достоинство для контрольных сумм и недостаток для паролей: современная видеокарта считает миллиарды хешей в секунду, то есть перебирает весь словарь за минуты.

Argon2id устроена иначе: она намеренно требует много памяти — в нашей настройке 64 МиБ на одну проверку. Видеокарта имеет много вычислительных ядер, но мало памяти на ядро, поэтому её преимущество исчезает. Перебор становится экономически невыгодным.

СОЛЬ — ЗАЧЕМ ОНА, ЕСЛИ ХЕШ И ТАК НЕОБРАТИМ

Соль — случайное значение, своё для каждого пароля, которое подмешивается перед хешированием. Без неё одинаковые пароли давали бы одинаковые хеши, и возникают две беды: во-первых, применима заранее посчитанная таблица «хеш → пароль» (радужная таблица), во-вторых, сразу видно, у каких пользователей пароли совпадают.

Соль не защищает от перебора одного пароля — от него защищает стоимость вычисления. Соль делает невозможным перебор **всех сразу** по готовой таблице.

Хеш хранится в формате РНС — это строка вида `$argon2id$v=19$m=65536,t=3,p=4$...`, где параметры стоимости записаны рядом с самим хешем. Смысл: через год можно поднять стоимость, и приложение при следующем успешном входе пересчитает хеш по новым параметрам, ничего не спрашивая у пользователя.

3.3. Шифрование данных: AES-256-GCM

Для конфиденциальных записей используется **симметричное** шифрование: один и тот же ключ и шифрует, и расшифровывает.

Важен не только алгоритм, но и **режим работы**. Режим GCM относится к классу **AEAD** — он даёт два свойства одновременно:

- **конфиденциальность** — данные нельзя прочитать без ключа;
- **целостность** — любую подмену шифротекста заметно, потому что вместе с данными хранится «тег аутентичности», короткая контрольная сумма на ключе.

Старый режим CBC давал только первое свойство: подменённый шифротекст расшифровался бы в мусор, но об этом никто бы не узнал. Поэтому выбран GCM.

Nonce: число, которое нельзя использовать дважды

Кроме ключа режиму нужен **nonce** — 12 случайных байт, свои для каждой операции шифрования. Название буквально означает «число, использованное один раз».

ПОЧЕМУ ПОВТОР NONCE — КАТАСТРОФА

Если один и тот же nonce использовать с одним ключом дважды, раскрывается «исключающее ИЛИ» двух открытых текстов: зная один, атакующий вычислит другой. Хуже того, компрометируется ключ аутентификации, и атакующий сможет подделывать теги, то есть выдавать поддельные данные за подлинные.

Поэтому в приложении при **редактировании** записи она перешифровывается заново с новым nonce. «Дописать» шифротекст принципиально нельзя.

AAD: привязка шифротекста к его месту

AAD (associated data, связанные данные) — это данные, которые не шифруются, но участвуют в вычислении тега целостности. Проще говоря: мы приклеиваем к шифротексту ярлык «это заголовок записи № 5, принадлежащей пользователю № 7, версия ключа 1», и любое несовпадение ярлыка ломает проверку.

Что это даёт: атакующий с доступом к базе не сможет взять шифротекст чужой записи и подставить в свою, чтобы приложение расшифровало его для него. Без AAD такая подмена прошла бы успешно — ключ-то один.

3.4. Один мастер-ключ, два подключа: HKDF

В приложении ключ нужен для двух разных задач: шифровать данные и считать слепой индекс для поиска. Использовать один и тот же ключ для двух задач нельзя — это общее правило криптографии.

HKDF — функция вывода ключей. Из одного мастер-ключа она получает сколько угодно независимых подключей, различающихся «меткой»:

```
K_enc = HKDF(мастер-ключ, метка "aes-gcm")
K_idx = HKDF(мастер-ключ, метка "blind-index")
```

Независимые означает: зная один подключ, невозможно вычислить другой или мастер-ключ. Практический смысл — если утечёт ключ индексации, данные останутся зашифрованными.

Сам мастер-ключ (32 байта) лежит в файле `/etc/pziis-vault/keys.env` с правами `0640 root:pziis`, вне кода и вне репозитория. Зашитые в код ключи — это отдельная слабость с номером CWE-798.

4. Слепой индекс: как искать в зашифрованном

Это самое интересное место работы, потому что здесь требование задания прямо противоречит требованию безопасности.

4.1. В чём противоречие

Задание требует «осуществить поиск конфиденциальных данных». Но правильно зашифрованные данные неотличимы от случайных байтов — искать по ним нечего. Два очевидных выхода оба плохи:

- **Расшифровывать всю таблицу на каждый запрос** — медленно, а главное опасно: весь открытый текст оказывается в памяти сразу.
- **Не шифровать поле, по которому ищем** — тогда и защиты нет.

4.2. Как работает слепой индекс

Идея: рядом с зашифрованной записью хранить не сам текст, а **отпечатки отдельных слов**, по которым можно сравнивать, но нельзя восстановить слово.

Отпечаток считается через **НМАС** — это хеш на ключе. Обычный хеш можно перебрать по словарю; НМАС без ключа перебрать нельзя, а ключ `K_idx` лежит в файловой системе, а не в базе.

Шаг	Что происходит
1	Заголовок «Договор аренды склада» приводится к единому виду: нижний регистр, разбиение на слова, отбрасывание слов короче трёх символов → <code>договор</code> , <code>аренды</code> , <code>склада</code>
2	Для каждого слова считается <code>НМАС-SHA256(K_idx, номер_владельца слово)</code> , от результата берутся первые 16 байт — это «тег»
3	Теги складываются в отдельную таблицу. Слов там нет — только 16-байтовые отпечатки
4	При поиске считается тег от поискового слова и отбираются записи с таким же тегом
5	Найденные записи расшифровываются, и совпадение подтверждается по открытому тексту — это отсеивает случайные совпадения из-за усечения тега

4.3. Что при этом всё-таки утекает — и это надо называть честно

Открытый текст не утекает. Но утекает **равенство и частота**: по базе видно, что две записи содержат одинаковое слово, и как часто слово встречается. Для словарных значений — скажем, названий организаций из короткого списка — по частоте можно догадаться, какое слово скрывается за каким тегом. Это называется частотным анализом.

Компромисс ограничен четырьмя мерами:

1. **Номер владельца входит в вычисление тега.** Одно и то же слово у разных пользователей даёт разные теги, поэтому частотный анализ не выходит за пределы одной учётной записи.

2. **Индексируется только заголовок.** Тело записи не индексируется вовсе, поэтому статистика не распространяется на весь текст.
3. **Тег усечён до 16 байт.** Редкие случайные совпадения дают ложные срабатывания, которые отсеиваются после расшифровки, — и дополнительно размывают статистику.
4. **Слова короче трёх символов не индексируются** — самые частотные короткие слова выпадают.

ФОРМУЛИРОВКА ДЛЯ ЗАЩИТЫ

Это то место работы, где безопасность прямо обменяна на функциональность. Правильно подавать так: «требование поиска несовместимо со стойким шифрованием в чистом виде; выбранный слепой индекс — измеримый компромисс с известной ценой, а не бесплатное решение». Названная цена звучит убедительнее, чем утверждение, что цены нет.

5. Как устроен вход и как работает сессия

5.1. Что такое сессия

HTTP не помнит, кто к нему обращался: каждый запрос независим. Чтобы приложение узнавало вошедшего пользователя, после успешного входа ему выдаётся **токен сессии** — длинная случайная строка, которую браузер сам присылает при каждом следующем запросе в виде cookie.

5.2. Два способа хранить сессию, и почему выбран первый

	Непрозрачный токен (выбран)	JWT
Что это	случайная строка, сама по себе ничего не значит	подписанный документ, в котором записано, кто пользователь
Где состояние	в базе на сервере	внутри самого токена
Как отозвать	удалить строку — действует мгновенно	нельзя до истечения срока подписи

Задание требует функцию деавторизации. С JWT «выход» был бы фикцией: токен остался бы действительным. Поэтому выбран непрозрачный токен, и выход из системы удаляет строку сессии на сервере.

Это проверяется в демонстрации напрямую: сохранить cookie, выйти, подставить cookie обратно — доступа нет.

5.3. Токен в базе хранится в виде хеша

В таблице сессий лежит не сам токен, а его SHA-256. Логика та же, что с паролями: если утечёт копия базы, по хешу токен не восстановить, значит чужую сессию не переиграть. Здесь достаточно быстрого SHA-256, а не Argon2, потому что токен — это 256 случайных бит, его нельзя угадать перебором в принципе.

5.4. Флаги cookie — три коротких слова с большим смыслом

Флаг	Что делает
HttpOnly	запрещает читать cookie из JavaScript. Даже если атакующий сумеет внедрить сценарий на страницу, украсть токен он не сможет
Secure	браузер не отправит cookie по незащищённому HTTP — только по HTTPS
SameSite=Strict	cookie не отправляется при переходе с чужого сайта. Это базовая защита от подделки запроса — см. раздел 6

5.5. Фиксация сессии

Атака выглядит так: злоумышленник заранее навязывает жертве известный ему идентификатор сессии, жертва входит под своим паролем — и идентификатор становится авторизованным. Защита простая: **при каждом входе выдавать новый** идентификатор. Старый, известный атакующему, авторизованным не станет никогда.

5.6. Защита от перебора пароля: три разных рубежа

Рубеж	Против чего	Как устроен
Лимит частоты на nginx	перебор с одного адреса	5 запросов в минуту на страницу входа
Лимит в приложении	то же, если запущено без прокси	тот же счётчик в памяти процесса
Блокировка учётной записи	распределённый перебор с многих адресов	10 неудач — блокировка на 15 минут

ПОЧЕМУ ЛИМИТ СТОИТ ИМЕННО НА NGINX, А НЕ ТОЛЬКО В ПРИЛОЖЕНИИ

Argon2 требует 64 МиБ памяти на одну проверку пароля. Значит сама форма входа — это ресурс: поток запросов на вход исчерпает оперативную память сервера. Поэтому поток надо отбивать **до** того, как он дойдёт до вычисления хеша.

Отсюда же следует, что лимит на адрес срабатывает раньше, чем порог блокировки: с одного адреса до десяти неудач дело не доходит. Блокировка учётной записи нужна против перебора со многих адресов — там лимит на адрес не помогает.

5.7. Почему текст ошибки всегда одинаковый

Сообщение «Неверный логин или пароль» одинаково и когда логина не существует, и когда пароль неверен, и когда запись заблокирована. Иначе форма входа превращается в средство **перечисления учётных записей**: атакующий сначала выясняет, какие логины существуют, и только потом подбирает к ним пароли.

По той же причине при неизвестном логине приложение всё равно считает хеш по фиктивному значению: если бы оно отвечало мгновенно, разница во времени ответа выдавала бы существующие логины даже при одинаковом тексте.

6. Атаки на веб-приложение и защита от них

6.1. SQL-инъекция

Суть. Приложение склеивает запрос к базе из строк, и данные пользователя становятся частью команды. Ввод `' OR 1=1 --` в поле поиска превращает условие «где имя равно введённому» в условие «где истина», и выдаётся вся таблица.

Неправильная защита — фильтровать кавычки. Способов обойти фильтр много, и рано или поздно один найдётся.

Правильная защита — параметризованные запросы. Текст запроса и значения передаются в базу **по отдельности**, на уровне протокола. База получает запрос со «пустыми местами» и отдельно список значений, и значение уже не может стать частью команды — независимо от того, что в нём написано.

6.2. XSS — внедрение сценария

Суть. Атакующий сохраняет в данных приложения текст вроде `<script>...</script>`. Когда другой пользователь открывает страницу, браузер считает это разметкой и выполняет сценарий — уже от имени жертвы, с её правами и её cookie.

В приложении три независимых рубежа:

1. **Автоэкранирование при выводе.** Шаблонизатор по умолчанию заменяет `<` на `<`, и браузер выводит текст как текст, а не как разметку.
2. **Политика CSP** — заголовок, которым сервер сообщает браузеру, что сценарии на этой странице запрещены вообще (`script-src 'none'`). В приложении своего JavaScript нет ни строчки, поэтому запрет ничего не ломает.
3. **Флаг `HttpOnly`** на cookie: даже сработавший сценарий не прочитает токен сессии.

6.3. CSRF — подделка запроса

Суть. Жертва, будучи авторизованной в нашем приложении, открывает чужую страницу. На той странице спрятана форма, отправляющая запрос на наш адрес. Браузер послушно добавляет cookie жертвы — и запрос выполняется от её имени. Пользователь ничего не заметил.

Защита двойная:

- **`SameSite=Strict`** — браузер не приложит cookie к запросу, инициированному с чужого сайта. Первый рубеж.
- **CSRF-токен** — случайное значение, связанное с сессией, которое вкладывается в каждую форму и проверяется на сервере. Чужая страница этого значения не знает и подобрать не может. Нужен потому, что `SameSite` поддерживается не всеми браузерами и не покрывает все случаи.

6.4. Утечка через сообщения об ошибках

Подробная ошибка полезна разработчику и полезна атакующему: она раскрывает версии, структуру таблиц, пути в файловой системе. Поэтому наружу уходит обобщённое сообщение, а подробности и трассировка — только в журнал сервера.

6.5. Сводка: что чему противостоит

Атака	Защита	CWE
SQL-инъекция	параметризованные запросы, роль без DDL, построчная защита	89
XSS	автоэкранирование, CSP, HttpOnly	79
CSRF	SameSite=Strict плюс CSRF-токен	352
Фиксация сессии	новый идентификатор при каждом входе	384
Перебор пароля	Argon2id, лимиты частоты, блокировка	307
Перечисление учётных записей	единый текст ошибки, выравнивание времени ответа	204
Кража копии базы	Argon2id, AES-256-GCM, хеши токенов	312
Перехват трафика	TLS, HSTS, флаг Secure	319
Излишние полномочия	минимальные привилегии роли и процесса	250
Утечка через ошибки	обобщённые сообщения наружу	209

7. Два контура контроля доступа

Права проверяются дважды и независимо. Это называется эшелонированной защитой.

7.1. Первый контур — в коде приложения

Обычная проверка роли: гостю нельзя писать, к конфиденциальным записям допускаются только роли «пользователь» и «администратор», к чужой записи не допускается никто. Проверка выполняется **до** чтения объекта, а не после.

7.2. Второй контур — в самой базе данных

Построчная защита (RLS) — механизм PostgreSQL, позволяющий задать правило: какие строки таблицы видны той или иной роли. Правило выполняет сама база, независимо от кода приложения.

Работает это так: в начале каждой транзакции приложение сообщает базе, от чьего имени она выполняется — записывает номер текущего пользователя в специальный параметр. Политика сравнивает владельца строки с этим номером.

ТРИ ВАЖНЫЕ ТОНКОСТИ

- **Параметр локален для транзакции.** Соединения к базе берутся из общего пула и переиспользуются между запросами. Если бы значение сохранялось в соединении, следующий запрос увидел бы данные предыдущего пользователя.
- **Безопасный отказ по умолчанию.** Если параметр не выставлен вовсе, сравнение не выполняется ни для одной строки и выборка пуста. Забытый вызов приводит к «ничего не видно», а не «видно всё».
- **Владелец таблицы не подчиняется RLS.** Поэтому владельцем таблиц назначен `postgres`, а приложение работает под отдельной ролью. Будь владельцем сама роль приложения, все политики оказались бы декорацией.

Практический смысл двух контуров: ошибка в коде или даже успешная инъекция не выдадут строки чужого владельца, потому что фильтрацию выполняет база.

7.3. Ограничение полномочий администратора

Администратор управляет учётными записями, ролями, блокировками, журналом аудита и неконфиденциальными данными. Но **открытый текст чужих конфиденциальных записей ему недоступен**: он видит только метаданные — владельца, время, размер.

Зачем такое ограничение: чтобы компрометация одной учётной записи не раскрывала данные всех пользователей. Это применение принципа разделения обязанностей — администрирование системы и доступ к содержимому данных разведены.

ЧЕСТНАЯ ОГОВОРКА, КОТОРУЮ НАДО ЗНАТЬ

Ограничение держится на коде, политике построчной защиты и представлении метаданных, но **не на криптографии**: ключ шифрования принадлежит сервису, поэтому сервис технически способен расшифровать любую запись. Криптографическая гарантия потребовала бы отдельного ключа на каждого пользователя, выведенного из его пароля. Это первая мера в списке усовершенствований.

8. Минимальные привилегии: в СУБД и в ОС

8.1. Роль приложения в базе данных

Приложение подключается к базе под отдельной ролью, у которой **нет** прав суперпользователя, создания баз, создания роли и обхода построчной защиты. Из привилегий выданы только операции с данными шести таблиц.

Отдельно про журнал аудита: на него выданы только `INSERT` и `SELECT`. Права изменять и удалять записи журнала не выдано намеренно — **скомпрометированное приложение не сможет затереть следы своих действий**. Права менять структуру базы (DDL) не выдано вовсе.

8.2. Процесс приложения в операционной системе

Мера	Что она предотвращает
Процесс работает от непривилегированного пользователя, а не от <code>root</code>	взлом приложения не даёт прав администратора системы
Код принадлежит <code>root</code> , приложение — только на чтение	взломанный процесс не перепишет собственный код и не оставит закладку
Ни одной специальной возможности (capability)	нечего использовать для повышения привилегий; порт 443 слушает прокси
Файловая система только для чтения, кроме каталога журналов	некуда положить посторонний файл
Ключи в файле с правами <code>0640</code> <code>root:pziis</code>	прочие пользователи системы их не прочитают

9. Защита канала связи

Что происходит без шифрования канала. Пароль и токен сессии идут по сети открытым текстом. Любой, кто видит трафик — на общем Wi-Fi, на промежуточном узле — прочитает их «снифером». Это простейшая и самая результативная атака.

TLS шифрует канал. В работе разрешены только версии 1.2 и 1.3 и только шифронаборы класса AEAD — по той же причине, что и для данных: нужна не только конфиденциальность, но и целостность.

Атака «человек посередине» — атакующий встаёт между клиентом и сервером и выдаёт себя за сервер. Защита — сертификат: браузер проверяет, что сервер тот, за кого себя выдаёт. Проверка опирается на подпись доверенного удостоверяющего центра.

ОГРАНИЧЕНИЕ СТЕНДА, КОТОРОЕ НАДО НАЗВАТЬ САМОМУ

На стенде сертификат **самоподписанный**. Это значит, что защита от «человека посередине» формально неполна: браузер не может проверить подлинность сервера и потому показывает предупреждение. Для учебного стенда допустимо, для эксплуатации нужен сертификат доверенного удостоверяющего центра. Об этом лучше сказать первым, чем услышать вопрос.

HSTS — заголовок, которым сервер говорит браузеру: «ко мне обращайся только по HTTPS, и запомни это на год». Он закрывает окно, когда первый запрос уходит по HTTP и может быть перехвачен.

Сетевой периметр. Наружу открыт единственный порт — 443 у прокси. Сервер приложений слушает только петлевой интерфейс, база данных — тоже. Их нельзя достичь снаружи в принципе, а не «по правилу межсетевого экрана».

10. Что реализовано

10.1. Требования задания и обработчики

№	Требование	Реализация
1	добавить пользователя в систему	POST /register — самостоятельная регистрация с ролью «гость»; POST /admin/accounts — создание администратором с назначением роли
2	авторизовать по идентификационным данным	POST /login — проверка по хешу Argon2id, выдача серверной сессии
3	CRUD и поиск конфиденциальных данных	/secrets/* — AES-256-GCM, поиск по слепому индексу; поиск отправляется методом POST, чтобы фраза не попала в журнал прокси и историю браузера
4	CRUD и поиск неконфиденциальных данных	/notes/* — открытое хранение, полнотекстовый поиск
5	деавторизовать пользователя	POST /logout — удаление строки сессии; POST /logout/all — закрытие всех сессий записи

10.2. Состав

Часть	Что содержит
Схема базы данных	роли уровня кластера (модель из работы №1 плюс сервисная роль), шесть таблиц, привилегии, политики построчной защиты, представление метаданных
Приложение	девять модулей: криптография, пул соединений с контекстом субъекта, сессии, политика пароля, два класса данных, журнал аудита, маршруты
Интерфейс	восемь серверных шаблонов; своего JavaScript нет вовсе
Развёртывание	прокси с TLS и лимитами, юнит systemd с ужесточением, правила межсетевого экрана, композиция из трёх контейнеров
Проверки	17 криптографических (без базы) и 39 сквозных — всего 56

10.3. Проверки нашли настоящий дефект

СИЛЬНЫЙ АРГУМЕНТ ДЛЯ ЗАЩИТЫ

Первый прогон проверок упал на CSRF: функция сравнения `hmac.compare_digest` отказывается сравнивать строки с символами вне ASCII. Присланный атакующим токен с кириллицей приводил к ответу **500 вместо 403** — то есть к необработанному исключению прямо на пути проверки защиты, на данных, которые атакующий контролирует полностью. Исправлено сравнением на байтах.

Механизм был спроектирован верно, а его реализация давала сбой. Это ровно тот случай, ради которого проверки и пишутся, и лучшее доказательство, что анализ был настоящим, а не декларативным.

10.4. Остаточные риски — что НЕ защищено

Риск	Почему остаётся
Частотный анализ слепого индекса	проектная цена за требование поиска; ограничен пределами одной учётной записи
Ключ и шифротекст на одном хосте	шифрование защищает от кражи копии базы или диска, но не от захвата машины
Сервис способен расшифровать любую запись	ограничение полномочий администратора логическое, а не криптографическое
Самоподписанный сертификат	защита от «человека посередине» неполна
Открытый текст в памяти процесса	Python не позволяет надёжно затирать строки — это тема лабораторной работы №3
Распределённый отказ в обслуживании	мерами одного хоста не отражается
Компрометация <code>root</code> , социальная инженерия	средствами приложения не закрывается и не заявляется

11. Вопросы и ответы

В. Чем анализ защищённости отличается от настройки прав?

О. Настройка описывает нормальное поведение системы. Анализ защищённости исходит из наличия злоумышленника и проверяет, чего он не сможет сделать. Методичка называет это подходом «чёрного ящика»: система рассматривается снаружи, как её видит атакующий.

В. Когда хешируют, а когда шифруют?

О. Если значение нужно будет показать — шифруют. Если достаточно уметь проверить совпадение — хешируют, потому что расшифровывать нечего. Поэтому пароли хешируются, а текст конфиденциальной записи шифруется.

В. Почему для паролей не подходит SHA-256?

О. Она спроектирована быстрой: видеокарта считает миллиарды хешей в секунду. Argon2id намеренно требует 64 МиБ памяти на проверку, что лишает видеокарту преимущества и делает перебор невыгодным.

В. Зачем соль, если хеш необратим?

О. Без соли одинаковые пароли дают одинаковые хеши: применима готовая таблица «хеш → пароль», и видно, у кого пароли совпадают. Соль не мешает перебору одного пароля — от него защищает стоимость вычисления, — но делает невозможным перебор всех сразу по таблице.

В. Что такое AEAD и почему не CBC?

О. AEAD — режим, дающий конфиденциальность и целостность одновременно: подмена шифротекста обнаруживается по тегу аутентичности. CBC давал только конфиденциальность, подмену не замечал и был уязвим к атакам на выравнивание.

В. Что будет, если повторить nonce?

О. Раскроется «исключающее ИЛИ» двух открытых текстов, и скомпрометируется ключ аутентификации, то есть можно будет подделывать теги. Поэтому при редактировании запись перешифровывается заново с новым nonce, а не правится частично.

В. Что такое AAD и что было бы без него?

О. Это данные, которые не шифруются, но участвуют в вычислении тега. Они привязывают шифротекст к конкретной записи, владельцу, версии ключа и полю. Без них атакующий с доступом к базе подставил бы шифротекст чужой записи в свою, и приложение расшифровало бы его для него.

В. Зачем выводить два подключа вместо одного ключа?

О. Один ключ для двух задач использовать нельзя. HKDF выводит из мастер-ключа независимые подключи, различающиеся меткой: зная один, невозможно вычислить другой. Практически: утечка ключа индексации не раскрывает зашифрованные данные.

В. Как искать по зашифрованным данным?

О. Через слепой индекс: для каждого слова заголовка считается HMAC на секретном ключе, усечённый до 16 байт, и складывается в отдельную таблицу. Поиск считает такой же отпечаток от запроса и сравнивает. Слов в базе нет, а восстановить их по отпечатку без ключа невозможно.

В. Что утекает через слепой индекс?

О. Равенство и частота слов в пределах одной учётной записи — для словарных значений возможен частотный анализ. Открытый текст не утекает. Ограничено четырьмя мерами: владелец входит в вычисление отпечатка, индексируется только заголовок, отпечаток усечён, короткие слова не индексируются.

В. Почему не JWT?

О. Задание требует деавторизации, а JWT нельзя отозвать до истечения срока подписи. С непрозрачным токеном выход удаляет строку на сервере, и доступ прекращается мгновенно.

В. Что дают флаги cookie?

О. `HttpOnly` — cookie недоступна из JavaScript, поэтому сработавший XSS не украдёт токен. `Secure` — cookie не уйдёт по HTTP. `SameSite=Strict` — cookie не приложится к запросу с чужого сайта, это базовая защита от CSRF.

В. Как защищаетесь от SQL-инъекций и почему не фильтруете кавычки?

О. Запросы параметризованы: текст запроса и значения идут в базу отдельно, на уровне протокола, поэтому значение не может стать частью команды. Фильтрация кавычек ненадёжна — её обходят. Дополнительно роль приложения не умеет DDL, а построчная защита ограничивает выборку.

В. Чем отличаются XSS и CSRF?

О. При XSS атакующий внедряет свой сценарий в нашу страницу и он выполняется у жертвы. При CSRF никакого внедрения нет: чужая страница просто отправляет запрос на наш адрес, а браузер сам прикладывает cookie жертвы. Первое лечится экранированием и CSP, второе — `SameSite` и CSRF-токеном.

В. Зачем построчная защита, если права уже проверены в коде?

О. Это второй, независимый контур. Ошибка в коде или успешная инъекция не выдадут строки чужого владельца, потому что фильтрует сама база. Если параметр текущего пользователя не выставлен, выборка пуста — безопасный отказ по умолчанию.

В. Почему администратор не видит чужие секреты? Это же противоречит заданию.

О. Задание требует полного доступа ко всем **подсистемам**, и он есть: учётные записи, роли, блокировки, аудит, неконфиденциальные данные, метаданные конфиденциальных записей. Ограничен доступ к **содержимому** чужих секретов, чтобы компрометация одной учётной записи не раскрывала данные всех пользователей.

В. А технически он действительно не может их прочитать?

О. Нет: ограничение логическое, а не криптографическое. Ключ принадлежит сервису, значит сервис способен расшифровать любую запись. Криптографическая гарантия требует отдельного ключа на пользователя, выведенного из его пароля, — это первая мера в списке усовершенствований.

В. Почему журнал аудита нельзя изменить приложением?

О. На таблицу журнала роли приложения выданы только `INSERT` и `SELECT`. Смысл: скомпрометированное приложение не сможет затереть следы. В журнал при этом пишутся только идентификаторы объектов — ни заголовков записей, ни поисковых фраз, иначе журнал стал бы вторым, незащищённым хранилищем секретов.

В. Что будет, если украдут копию базы?

О. Пароли не восстановят (Argon2id с солью), сессии не переиграют (в базе только хеши токенов), конфиденциальные записи не прочитают (ключ в файловой системе, не в базе), подмену записи обнаружат (тег целостности). Утекут частотная статистика слов в пределах учётной записи и метаданные: количество записей, размеры, время изменения.

В. Что будет при компрометации `root`?

О. Раскроется всё: `root` прочитает файл с ключом и память процесса. Защита от вредоносного администратора системы средствами приложения невозможна и не заявляется. Шифрование защищает от кражи копии базы, резервной копии или диска — не от захвата машины.

В. Что вы не защитили?

О. Компрометацию `root`, распределённый отказ в обслуживании, социальную инженерию, уязвимости в самих зависимостях. Сертификат на стенде самоподписанный, поэтому защита от «человека посередине» формально неполна. Открытый текст неизбежно присутствует в памяти процесса — это предмет лабораторной работы №3.

12. Итог

Семантическая составляющая. Защищённость приложения не является свойством одного механизма. Она складывается из согласованных решений на четырёх независимых уровнях: права в операционной системе, привилегии и политики в СУБД, конфигурация сетевого периметра, криптографическая обработка самих данных. Через все уровни проходят три принципа. **Минимизация привилегий:** процесс не владеет своим кодом, роль СУБД не умеет менять структуру, приложение не может стереть журнал аудита, администратор не читает открытый текст чужих секретов. **Эшелонирование:** контроль доступа реализован дважды и независимо, поэтому одиночная ошибка не раскрывает данные. **Называние цены:** безопасность обменивается на функциональность, и обмен нужно проговаривать вслух — требование поиска по конфиденциальным данным несовместимо со стойким шифрованием в чистом виде, а слепой индекс есть измеримый компромисс с известной ценой.

Прагматическая составляющая. Получен воспроизводимый и проверяемый результат: схема данных с разделением классов информации, ролевая модель, согласованная с настройкой из первой работы, набор конкретных конфигураций прокси, службы и межсетевого экрана, а также матрица «атака → мера → остаточный риск», привязанная к перечню CWE. Каждое утверждение о защищённости сопровождается способом его проверки: 56 автоматических проверок и 12 сценариев демонстрации превращают раздел анализа из декларации в исполняемый чек-лист. Практическая ценность такого подхода подтвердилась сразу — первый прогон вскрыл дефект в сравнении CSRF-токенов, при котором верно спроектированный механизм отвечал

ошибкой 500 на контролируемые атакующим данные. Одновременно зафиксированы границы применимости: защита не распространяется на компрометацию `root`, распределённый отказ в обслуживании и социальную инженерию.